

4주차 · Simulink PID 와 첫 제어기

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-24

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 본 과목 담당 교수가 직접 강의한 것이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

#	과목	수준	언어	영상	자료·코드
1	제어공학 — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	제어시스템설계 — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	캡스톤디자인 — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	제어공학특론 — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	센서신호처리 및 융합 — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB-Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것. 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`

저장소	주소	받는 자리 (WSL)
강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트	https://github.com/wkyouncnu/Capstone-Design	<code>~/Capstone-Design</code>
ROS 2 예제 패키지 <code>usv_basics</code> — 2주차부터 쓰는 노드 코드	https://github.com/wkyouncnu/usv_basics	<code>~/capstone_ws/src/usv_basics</code>

연제	명령 (VS Code 의 WSL 창 터미널)	하는 일
처음 한 번	<code>cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git</code>	강의자료 전체를 받음 (약 400 MB)
처음 한 번	<code>mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git</code>	예제 패키지를 받음. 이어서 <code>cd ~/capstone_ws && colcon build --symlink-install</code>
매주 수업 전	<code>cd ~/Capstone-Design && git pull</code> · <code>cd ~/capstone_ws/src/usv_basics && git pull</code>	바뀐 파일만 받음. 다시 clone 하지 않음
무엇이 바뀌었는지	<code>git log --oneline -10</code> · <code>git show --stat HEAD</code>	교수가 갱신한 내역을 확인

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엮
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → [강의자료는-한-번-받고-git-pull-로-갱신한다](#)

- 과목: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- 이번 주차 학습 내용: 제어를 배가 아닌 가장 순한 플랜트에서 원리부터 익히고, 같은 제어를 오프라인 WAM-V 의 속도 → 헤딩 → 내부 루프로 옮긴 뒤 같은 계인을 VRX 에서 확인함

★ 시작 전 확인

- 강의자료부터 갱신: `cd ~/Capstone-Design && git pull` ([강의자료는-한-번-받고-git-pull-로-갱신한다](#), 처음 받는 법은 2주차 2-6)
- MATLAB R2024b + Simulink + ROS Toolbox 설치 확인
- 1~4단계는 Gazebo 없이 MATLAB 만으로 됨. VRX 는 5단계(실습 N 절부터)에서만 필요함
- VRX 기동은 3주차 2-3 의 경량 모드(`w03_vrx_lite/run_vrx.sh`)를 씀
- Simulink 가 처음이면 [A1 Simulink 기초](#) 를 먼저 할 것 — 창·블록·버스·MATLAB Function 사용법은 그 부록에 있고, 이번 주차는 그것을 이미 안다고 전제함

I 이번 주차의 순서 — 다섯 단계

단계	내용	플랜트	Gazebo	실습
1	PID 기본 원리	질량-스프링-댐퍼 (배 아님)	불필요	B~G
2	속도 제어	오프라인 WAM-V 종방향	불필요	H-I
3	헤딩 제어	오프라인 WAM-V 요측	불필요	J~L
4	속도 + 헤딩 내부 루프	오프라인 WAM-V 3자유도	불필요	M
5	같은 알고리즘을 VRX 로	Gazebo + ROS 2	필요	N~T

- 각 단계는 **제어기 없이 돌려 보는 것(개루프)**으로 시작함. 제어 대상이 무엇을 하는지 먼저 재고 그 숫자로 계인을 정함
- 2~4단계와 5단계는 **쌍둥이임**. 제어기 상자와 계인이 같고 플랜트 자리만 바뀜

이 주차를 마치면 할 수 있어야 하는 것

1. 자유물체도에서 **운동방정식** → 전달함수를 유도하고 $\zeta \cdot \omega_n$ 을 계수로 쓰기
2. **P · I · D** 가 각각 무엇을 하고 무엇을 못 하는지 스텝응답 수치로 설명하기
3. PID 를 **Gain · Integrator · Sum** 으로 직접 조립하고 라이브러리 블록과 같음을 확인하기
4. **유사미분**과 **안티와인드업**이 왜 필요한지 설명하고 컨 경우·곤 경우를 수치로 비교하기
5. 개루프 계단 추력에서 **정상상태 속도 · 시상수 · 추력 한계 최고속**을 재고 손계산과 맞추기
6. 속도 루프에 **왜 I**가 필요하고 **D**가 해로운지 운동방정식으로 설명하기
7. 개루프 계단 모멘트에서 **선수각이 멈추지 않는 이유**를 설명하기
8. 요측 운동방정식에서 **P-D 극배치** 공식을 유도하고 계인을 계산하기
9. $\pm 180^\circ$ **이음매**에서 **ssa**가 없으면 무슨 일이 생기는지 수치로 보이기
10. Simulink 의 **ROS 2 Publish / Subscribe** 블록으로 Gazebo 와 데이터를 주고받기
11. **오프라인 결과와 VRX 결과를 대조**하고 차이가 나는 항목을 지목하기

준비물

항목	내용
환경	1~3주차 WSL2 + ROS 2 + VRX (5단계에서만)
MATLAB	R2024b + Simulink + ROS Toolbox
배포 파일	W04_simulink/ (모델 14개 + 스크립트 7개)
파라미터	W04_setup.m 하나 — 1~5단계의 모든 값이 여기 있음

1부 · 이론

각 절은 **현상** → **결과 식** → **유도** 순서임. 실측 표는 2부의 해당 실습 절에 있고, 1부는 그 숫자 중 요점만 인용함

1-1. 개루프와 피드백 — 왜 되먹임이 필요한가

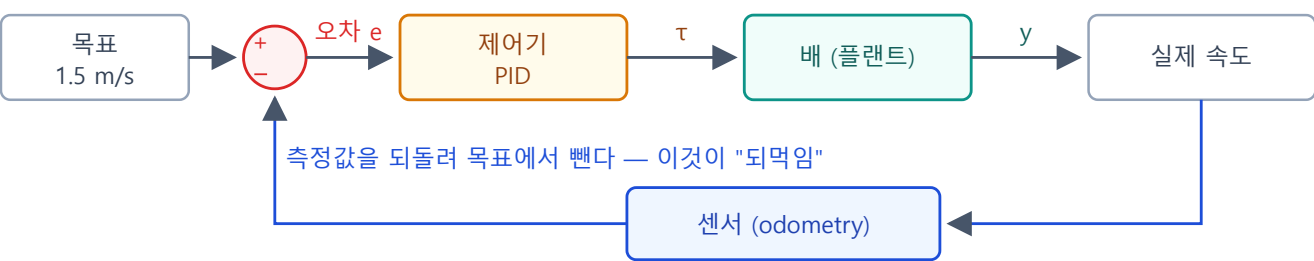
★ **제어기가 하는 일은 한 문장**
재보고, 틀린 만큼 고침. 재보지 않으면 제어가 아니라 단순한 명령임

개루프 — 재보지 않고 내보낸다



조류·바람·짐의 무게가 달라지면 그만큼 어긋난 채로 끝난다. 얼마나 어긋났는지 알 방법도 없다.

피드백(폐루프) — 재보고, 틀린 만큼 고친다



자리	하는 일
맨 왼쪽 목표값	얼마로 가고 싶은가
동그라미 + -	목표값에서 측정값을 뺌 → 오차
주황 상자 (제어기)	오차를 보고 제어입력을 만들
청록 상자 (플랜트)	제어입력을 받아 실제로 움직이는 것
파랑 상자 (센서)	실제 값을 재서 되돌려 줌

	개루프	피드백(폐루프)
측정	안 함	함
판단 근거	미리 정한 값	지금의 오차
외란(조류·바람·짐)	그만큼 어긋난 채 끝남	어긋난 만큼 스스로 메움
대가	없음	잘못 설계하면 진동하거나 발산함

- 3주차의 "추력 200 N 을 준다"는 개루프였음. 배가 몇 m/s 로 가는지 보지 않았음
- 이번 주차는 **속도를 재서** 목표와 비교함. 다만 각 단계를 **개루프로 먼저** 돌려 봄 — 제어 대상이 무엇을 하는지 모르면 계인을 정할 근거가 없기 때문임

1 피드백이 공짜가 아닌 이유

되먹임은 고칠 힘을 주지만 동시에 **되돌아오는 길**을 만듦. 너무 세게 고치면 고친 것이 다시 오차가 되어 돌아옴 → 진동

그래서 "얼마나 세게 고칠 것인가"를 정하는 값이 필요함. 그것이 **게인**

1-2. 제어기를 배가 아닌 것에서 먼저 배우는 이유

- 배는 게인 하나를 바꾸면 **여러 가지가 같이 움직임** — 항력, 추진기 포화, 두 루프의 간섭
- 무엇 때문에 달라졌는지 분리되지 않으면 배울 수 없음. 자전거를 배우려고 오토바이에 올라타지 않는 것과 같음
- 1단계의 플랜트는 **질량-스프링-댐퍼** 하나 ($m = 1 \text{ kg}$, $b = 2 \text{ N}\cdot\text{s/m}$, $k = 2 \text{ N/m}$). 입력은 힘 [N], 출력은 위치 [m]이며 제어기 없이 두면 살짝 출렁이다 멈춤

△ 1단계의 y 는 동쪽 위치가 아님

3주차부터 y 는 동쪽 위치였음. 1단계에서만 y 는 **수레의 위치**, 즉 일반적인 출력임
배로 돌아가는 1-13 절부터는 다시 3주차 규칙(x 북 · y 동, 코드 x_n · y_n)을 씀

1-3. 플랜트를 적는 법 — 그림에서 전달함수까지

- **현상**: 수레를 1 N 으로 계속 밀면 끝없이 밀려나지 않고 **0.5 m 에서 멈춤** (실습 B 절 실측 0.50003 m)
- 밀려난 만큼 스프링이 되밀어 두 힘이 같아지는 자리에서 멈추기 때문임. 그 자리를 손으로 구할 수 있어야 제어기를 설계할 수 있음

$$m\ddot{y} + b\dot{y} + ky = f \quad \Rightarrow \quad G(s) = \frac{Y(s)}{F(s)} = \frac{1}{ms^2 + bs + k} = \frac{1}{s^2 + 2s + 2}$$

유도 — 한 줄씩

1. 수레에 걸리는 힘은 셋임 — 미는 힘 f , 스프링이 되미는 힘 $-ky$, 감쇠가 막는 힘 $-b\dot{y}$
 2. 뉴턴 제2법칙을 적음 — $\sum F = m\ddot{y}$
 3. 부호를 붙여 대입 — $f - ky - b\dot{y} = m\ddot{y}$
 4. 미분 항을 왼쪽으로 모음 — $m\ddot{y} + b\dot{y} + ky = f$
 5. 라플라스 변환을 취함 (초기값 0). 미분 한 번이 s 한 개임 — $y \rightarrow Y$, $\dot{y} \rightarrow sY$, $\ddot{y} \rightarrow s^2Y$
 6. 항별로 바꿔 적음 — $ms^2Y + bsY + kY = F$
 7. Y 로 묶음 — $(ms^2 + bs + k)Y = F$
 8. 양변을 F 와 $(ms^2 + bs + k)$ 로 나누면 위 결과식이 나옴
- **전달함수는 "입력을 넣으면 출력이 이렇게 나온다는 약속"** 을 s 의 분수 하나로 적은 것임. 자판기와 같이 속을 덮어 두고 넣은 것과 나온 것의 관계만 적음
 - 새 물리가 아님. 4번 식과 8번 식은 **같은 것을 두 가지로 적은 것임** (실습 B 절에서 두 줄이 겹치는 것으로 확인)
 - **정상상태 확인**: 멈춘 뒤에는 $\dot{y} = \ddot{y} = 0$ 이므로 $ky = f \rightarrow y_{ss} = f/k = 0.5 \text{ m}$. 현상과 맞음

1-4. 두 개의 다이얼 — ζ 와 ω_n

- **현상**: **zeta** 를 0.2 로 두고 Run 하면 오버슈트 55.5 %, 1 로 두면 0 % 임 (실습 B-4)
- ζ 는 속업소버의 세기, ω_n 은 엔진의 힘에 해당함

$$\frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \quad \Rightarrow \quad \omega_n = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{b}{2\sqrt{mk}}$$

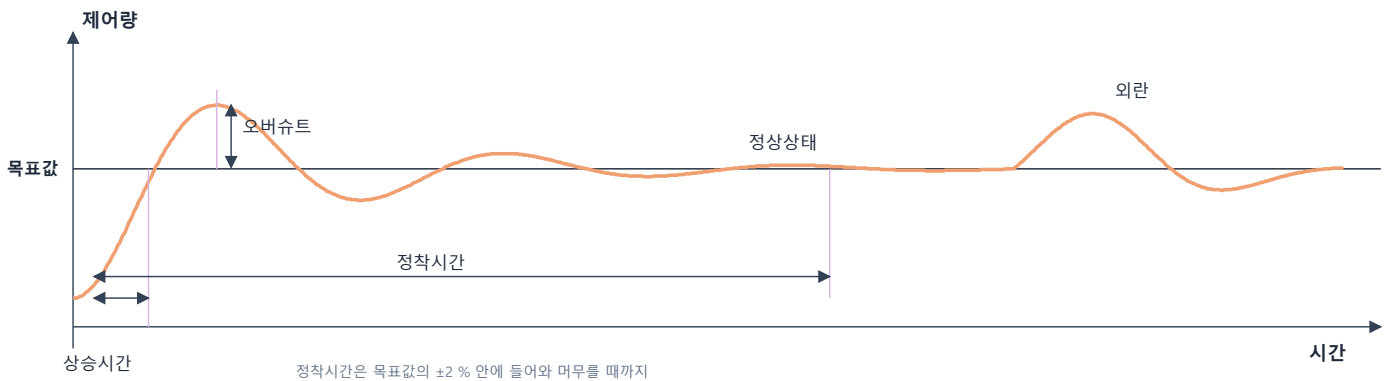
유도 — 한 줄씩

1. 2차계의 **표준형**을 적음. 분모의 계수 두 개가 곧 $\zeta\omega_n$ 임
 2. 표준형과 같은 꼴로 만들려고 1-3 의 분모를 m 으로 나눔 — $s^2 + (b/m)s + k/m$
 3. 계수를 하나씩 맞대어 놓음 — s^0 자리는 $\omega_n^2 = k/m$, s^1 자리는 $2\zeta\omega_n = b/m$
 4. 위 줄에서 $\omega_n = \sqrt{k/m}$
 5. 아래 줄에 그 ω_n 을 넣고 ζ 로 풀 — $\zeta = b/(2m\omega_n) = b/(2\sqrt{mk})$
 6. 값을 대입 — $\omega_n = \sqrt{2} = 1.4142 \text{ rad/s}$, $\zeta = 2/(2\sqrt{2}) = 0.7071$
- 오버슈트는 ζ 만으로 정해짐. 유도는 참조 강의 1 에 있고 본 과목은 **결과를 실측으로 확인함**

$$M_p = 100 e^{-\pi\zeta/\sqrt{1-\zeta^2}} [\%]$$

- ω_n 은 빠르기만 정함 — 두 배로 하면 상승·정착이 **정확히 절반**이 되고 오버슈트는 4.32 % 로 그대로임 (실습 B-4)

1-5. 응답을 판정하는 다섯 숫자



용어	뜻
상승시간	목표의 10 % 에서 90 % 까지 가는 데 걸린 시간
첨두시간	최고값에 닿는 시각
오버슈트	최종값을 얼마나 넘어섰는가 [%]
정착시간	최종값의 $\pm 2\%$ 안에 들어와 머무를 때까지
정상상태 오차	충분히 기다린 뒤에도 남는 목표와의 차이

정착시간이 왜 $4/(\zeta\omega_n)$ 인가

1. 2차계의 진동은 포락선 $e^{-\zeta\omega_n t}$ 안에서 줄어듦
 2. 포락선이 2 % 로 줄어드는 시각을 t_s 라 하면 $e^{-\zeta\omega_n t_s} = 0.02$
 3. 양변에 자연로그 — $-\zeta\omega_n t_s = \ln 0.02 = -3.91$
 4. $t_s = 3.91/(\zeta\omega_n) \approx 4/(\zeta\omega_n)$
- 계산: $\zeta = 0.7071$, $\omega_n = 1.4142 \rightarrow t_s \approx 4.00 \text{ s}$. 측정 4.22 s

! 외란은 스텝응답이 아니라 끝난 뒤에 음

그림 오른쪽에서 곡선이 한 번 솟았다 가라앉는 것이 외란임. 목표는 그대로인데 파도·조류가 배를 밀어낸 것 되먹임이 그것을 스스로 메우는지가 제어기의 진짜 시험

1-6. PID — 오차 하나를 세 가지로 읽는다

$$e(t) = y_d(t) - y(t), \quad \tau(t) = \underbrace{K_p e}_{\text{지금}} + \underbrace{K_i \int_0^t e d\sigma}_{\text{과거}} + \underbrace{K_d \dot{e}}_{\text{앞으로}} \implies C(s) = K_p + \frac{K_i}{s} + K_d s$$

갈래	무엇을 읽는가	비유	혼자서는 안 되는 이유	절
P	지금 얼마나 틀렸는가	스프링	오차가 0 이면 힘도 0. 오차가 남아야만 힘이 유지됨	1-7
I	얼마나 오래 틀리고 있는가	참을성 있는 손	과거를 쌓으므로 늦게 반응하고 늦게 멈춤	1-9
D	얼마나 빨리 좁혀지고 있는가	속업소버	변화만 보므로 어디에 있는지 모름	1-8

⚠ 제어 교과서의 r 과 u 를 본 과목에서는 쓰지 않음

이 과목에서 r 은 요각속도, u 는 전후 속도임 — 이번 주차의 속도 루프가 $u_{\text{ref}} - u$ 를 쓴 한 페이지에서 u 가 속도이기도 하고 제어입력이기도 하면 읽을 수가 없음
 그래서 목표값은 y_d , 제어입력은 τ 로 씀 (참조 강의 4-5 및 Fossen 과 같은 표기)
 배로 넘어가는 1-13 절부터는 제어기 출력을 전후력 $X \cdot$ 요 모멘트 N 으로 적음

1-7. P 가 할 수 있는 것과 없는 것

- 현상: K_p 를 $2 \rightarrow 10 \rightarrow 50$ 으로 키우면 빨라지고 더 출렁이며, 목표에 못 미친 채 멈춤 (남는 오차 0.500 / 0.167 / 0.0385, 실습 C-3)

$$y_{ss} = \frac{K_p}{k + K_p} y_d, \quad e_{ss} = \frac{k}{k + K_p} y_d, \quad \omega_n = \sqrt{\frac{k + K_p}{m}}, \quad \zeta = \frac{b}{2\sqrt{m(k + K_p)}}$$

유도 — 한 줄씩

- P 제어기는 $\tau = K_p (y_d - y)$ 임. 1-3 의 $m\ddot{y} + b\dot{y} + ky = \tau$ 에 대입
 - $m\ddot{y} + b\dot{y} + ky = K_p (y_d - y)$
 - y 가 든 항을 왼쪽으로 모음 — $m\ddot{y} + b\dot{y} + (k + K_p)y = K_p y_d$
 - K_p 가 k 옆에 붙었음. P 제어는 스프링을 하나 더 다는 것과 같음
 - 멈춘 뒤를 봄. 정상상태이므로 $\dot{y} = \ddot{y} = 0 \rightarrow (k + K_p)y_{ss} = K_p y_d$
 - 양변을 $(k + K_p)$ 로 나눔 — $y_{ss} = K_p y_d / (k + K_p)$
 - 오차는 $e_{ss} = y_d - y_{ss} = k y_d / (k + K_p)$ — 스프링이 있는 한 0 이 되지 않음
 - 응답 모양은 1-4 의 공식에 $k \rightarrow k + K_p$ 를 넣으면 나옴
- 검산 — $K_p = 2 / 10 / 50$ 에서 $y_{ss} = 0.500 / 0.833 / 0.962$, $\zeta = 0.500 / 0.289 / 0.139 \rightarrow M_p = 16.3 / 38.6 / 64.5$ %. 측정과 소수 첫째 자리까지 맞음 (실습 C-3)

★ P 만으로는 남는 오차를 없앨 수 없음

힘이 오차에 비례하므로 힘을 계속 내려면 오차가 남아 있어야 함

K_p 를 키우면 오차는 줄지만 0 이 되지 않고, 대신 ζ 가 $1/\sqrt{K_p}$ 로 작아져 오버슈트가 먼저 터짐

1-8. D 는 브레이크

- 현상: K_d 를 $0 \rightarrow 2 \rightarrow 6$ 으로 키우면 오버슈트가 $38.8 \rightarrow 16.9 \rightarrow 8.60$ % 로 깎이는데 도착점은 0.833 에서 한 자리도 변하지 않음 (실습 C-3)

$$m\ddot{y} + (b + K_d)\dot{y} + (k + K_p)y = K_p y_d \quad \Rightarrow \quad \zeta = \frac{b + K_d}{2\sqrt{m(k + K_p)}}$$

유도 — 한 줄씩

1. PD 제어기는 $\tau = K_p e + K_d \dot{e}$ 임
2. 목표값이 상수인 동안에는 $\dot{e} = \dot{y}_d - \dot{y} = -\dot{y}$
3. 대입하면 $\tau = K_p(y_d - y) - K_d \dot{y}$
4. 운동방정식에 넣고 $y\dot{y}$ 항을 왼쪽으로 모으면 위 결과식이 나옴
5. K_d 가 b 옆에 붙었음. D 제어는 댐퍼를 하나 더 다는 것과 같음
6. 1-4 의 공식에 $b \rightarrow b + K_d$ 를 넣으면 ζ 가 나옴
7. 정상상태에서는 $\dot{y} = 0$ 이라 K_d 항이 사라짐 $\rightarrow y_{ss}$ 는 1-7 과 같음

$\zeta > 1$ 인데도 넘치는 이유 — 영점

- $K_d = 6$ 이면 $\zeta = 8/(2\sqrt{12}) = 1.155$ 이므로 손계산은 오버슈트 0 을 예측하는데 측정은 8.60 % 임
- 이유는 PD 제어기가 분자에도 s 를 넣기 때문임

$$\frac{Y(s)}{Y_d(s)} = \frac{K_d s + K_p}{ms^2 + (b + K_d)s + (k + K_p)} \quad \Rightarrow \quad \text{영점 } s = -\frac{K_p}{K_d} = -1.67$$

- 영점은 응답의 앞부분을 끌어올려 오버슈트를 더함. K_d 가 클수록 영점이 원점에 가까워져 영향이 커짐
- ζ 만으로 오버슈트를 예측하면 PD 에서는 어긋남. 그래서 실측표가 필요함

1-9. I 는 참을성 있는 손

- 현상: $K_i = 4$ 를 켜면 남는 오차가 $0.167 \rightarrow 0.0043$ 으로 사라지고, 적분항이 2 N 에서 스스로 멈춤 (실습 C-3)
- 그 2 N 은 아무도 알려 주지 않았음. 제어기가 찾아낸 값임

왜 오차가 0 이 되는가 — 세 줄

1. 적분기는 입력이 0 일 때만 값이 변하지 않음
2. 응답이 멈췄다는 것은 적분기도 멈췄다는 뜻이므로 그 지점에서 $e = 0$
3. 그때 쌓여 있는 값이 곧 스프링을 이기는 데 필요한 힘 $k y_d = 2 \text{ N}$

너무 키우면 불안정해진다 — Routh 판별

1. PID 제어입력 $\tau = K_p e + K_i \int e + K_d \dot{e}$ 를 운동방정식에 넣고 양변을 한 번 더 미분하면 적분이 사라져 3차식이 됨
2. 특성방정식은 $ms^3 + (b + K_d)s^2 + (k + K_p)s + K_i = 0$
3. Routh 표를 만들 — 1행에 짝수 차수 계수, 2행에 홀수 차수 계수를 차례로 적음

차수	첫 열	둘째 열
s^3	m	$k + K_p$
s^2	$b + K_d$	K_i
s^1	$\frac{(b + K_d)(k + K_p) - mK_i}{b + K_d}$	0
s^0	K_i	—

4. s^1 행은 위 두 행의 어긋난 곱을 빼서 아래 행의 첫 값으로 나눈 것임
5. 안정 조건은 첫 열의 부호가 한 번도 바뀌지 않는 것 $\rightarrow K_i > 0$ 이고 $(b + K_d)(k + K_p) - mK_i > 0$
6. K_i 로 풀면

$$0 < K_i < \frac{(b + K_d)(k + K_p)}{m} = \frac{(2 + 4)(2 + 10)}{1} = 72$$

- 실습에서 쓰는 $K_i = 12$ 는 한계의 6분의 1 이라 안정함. 실습 C-4 에서 $K_i = 50$ (한계의 0.7 배)과 비교함

⚠ I 는 공짜가 아님

$K_i = 4$ 는 오차를 없애는 대신 정착시간을 1.32 → 4.84 초로 늘리고, $K_i = 12$ 는 빨리 메우는 대신 오버슈트를 4.69 % 로 되살림

1-10. 실제 미분 — 잡음과 계단 모서리

★ 미분은 크기가 큰 신호가 아니라 빠른 신호를 키움

센서 잡음이 신호 중에서 가장 빠름. 교과서의 PID 를 그대로 구현하면 여기서 먼저 부러짐

- **현상 ①**: 잡음이 섞인 $\sin(0.5t)$ 를 미분하면 참값 0.50 인데 순수 미분은 **109.91** 까지 튼 (유사미분은 1.73, 실습 E-3)
- 이유 한 줄 — 미분하면 **진폭에 ω 가 곱해짐**. 잡음은 ω 가 가장 큰 성분임

$$\frac{d}{dt} A \sin \omega t = A \omega \cos \omega t$$

필터를 붙이면 증폭에 천장이 생긴다 — 유도

1. 유사미분의 전달함수는 $D(s) = K_d N_f s / (s + N_f)$ 임
 2. 주파수 응답을 보려고 $s = j\omega$ 를 넣음 — $D(j\omega) = K_d N_f j\omega / (j\omega + N_f)$
 3. 느린 성분 ($\omega \ll N_f$): 분모가 N_f 에 가까움 → $D \approx K_d j\omega$. 즉 **미분 그대로**
 4. 빠른 성분 ($\omega \gg N_f$): 분모가 $j\omega$ 에 가까움 → $D \approx K_d N_f$. 즉 $K_d N_f$ 에서 **멈춤**
- 잡음을 없애는 것이 아니라 **증폭에 상한을 두는 것**임. 그래서 유사미분도 참값 0.50 보다 큰 1.73 이 나옴
 - N_f 는 "어디까지를 신호로 볼 것인가"를 정하는 값이며 본 과목은 10~20 을 씀. 200 으로 풀면 제어입력의 떨림이 **11.6 배** 가 됨 (실습 E-3)
 - 저역통과 필터 단독 실습은 부록 → [A1_Simulink_기초](#) ([w04_P4_lowpass](#) , 통과율 손계산 0.4472 대 측정 0.4466)

미분 킥 — 목표값이 계단이면 제어입력이 한 번 크게 튜다

1. 목표값이 크기 Δ 의 계단으로 바뀌면 오차도 같은 순간에 Δ 만큼 뛴
 2. 순수 미분이라면 그 순간의 미분값은 무한대임
 3. 유사미분은 위의 천장에 걸리므로 $K_d N_f \Delta$ 로 **막힘** — $4 \times 20 \times 1 = 80$
 4. P 항 $K_p \Delta = 10$ 을 더하면 90. 측정 **89.7** 과 맞음 (실습 E-3)
- 고치는 법은 플랜트가 아니라 **목표값을 부드럽게** 만드는 것임 ($1/(T_{ref}s + 1)$ 을 통과시킴). 킥이 **13 배** 줄고 정상상태 제어입력은 2.0 으로 같음

1-11. 실제 적분 — 포화와 되감기

★ 액추에이터에는 한계가 있고, 적분기는 그것을 모름

이 두 문장이 안티와인드업의 전부임

- **현상**: 닿을 수 없는 목표를 준 뒤 목표를 낮춰도 한참 돌아오지 않음. 오버슈트 28.7 % 대 0.03 % (실습 F-2)
1. 목표가 멀어 제어가 큰 값을 요구함
 2. 포화 블록에서 잘려 **실제로는 한계값만** 나감
 3. 그동안에도 오차는 남아 있으므로 **적분기는 계속 쌓임**

4. 목표에 도달해 오차가 0 이 되어도 쌓인 적분값이 계속 밀어붙임
5. 크게 지나친 뒤에야 되돌아옴

$$\dot{I} = K_i e + K_b (\tau_{\text{sat}} - \tau_{\text{raw}})$$

기호	뜻	PID_byhand 안의 이름
τ_{raw}	제어기가 원했던 제어입력	tau_raw
τ_{sat}	포화를 지나 실제로 나간 제어입력	tau_sat
K_b	되감기 이득	Kb_gain

- 포화 중에는 두 값이 달라 그 차이가 적분기를 한계 반대쪽으로 끌어냄. 포화가 풀리면 차이가 0 이 되어 평소의 PID 로 돌아옴

되감기가 멈추는 자리 I^* — 유도

1. 포화가 계속되는 동안 적분값이 어디에 머무는지를 구함. 더 이상 변하지 않으므로 $\dot{I} = 0$
 2. 상한 포화 중이므로 $\tau_{\text{sat}} = \tau_{\text{max}}$ 이고, $\tau_{\text{raw}} = P + I^* + D$
 3. 식에 넣음 — $0 = K_i e + K_b (\tau_{\text{max}} - P - I^* - D)$
 4. I^* 로 풀 — $I^* = \tau_{\text{max}} - P - D + (K_i/K_b) e$
 5. 숫자를 넣음. 계단 직후 $e = 1$, $P = 10$, $D \approx 0$, $\tau_{\text{max}} = 2.5$, $K_i/K_b = 4 \rightarrow I^* = 2.5 - 10 + 4 = -3.5$
- 적분값이 음수로 내려가 있음. 고장이 아니라 식이 그렇게 시킨 것임
 - 포화가 풀리는 순간 적분기가 밀어붙일 것을 미리 없애 두는 것이 되감기의 역할임. $K_b = 0$ 이면 3번 식의 둘째 항이 사라져 적분기가 그대로 쌓임

1-12. 계인을 정하는 순서

★ 세 개를 동시에 만지지 않음. 한 번에 하나씩

1. $K_i = 0$, $K_d = 0$ 으로 두고 K_p 만 올림 — 출렁이기 시작하는 지점 직전까지
 2. K_d 를 더해 오버슈트를 깎음. 제어입력이 떨리기 시작하면 N_f 를 낮춤
 3. K_i 를 조금씩 더해 남는 오차를 없앴. 정착시간이 늘어나면 너무 많이 넣은 것
 4. 포화가 있으면 되감기를 켜 ($K_b > 0$)
- 단계별 실측은 실습 G 절에 있음. ③ 에서 오버슈트가 $0 \rightarrow 28.7\%$ 로 되돌아가고 ④ 가 그것을 되돌림

오버슈트에서 ζ 를 거꾸로 읽는 법

1. $M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$ (M_p 는 백분율이 아니라 0~1 의 비)
2. 양변에 자연로그 — $\ln M_p = -\pi\zeta/\sqrt{1-\zeta^2}$
3. 양변을 제곱 — $\ln^2 M_p (1 - \zeta^2) = \pi^2 \zeta^2$
4. ζ^2 로 묶음 — $\ln^2 M_p = \zeta^2 (\pi^2 + \ln^2 M_p)$

$$\zeta = \frac{-\ln M_p}{\sqrt{\pi^2 + \ln^2 M_p}}$$

- 예: 측정 오버슈트 14.2 % $\rightarrow M_p = 0.142 \rightarrow \zeta = 0.55$

🔧 PID Tuner 는 출발점을 주는 도구

PID 블록을 더블클릭하고 **Tune** 을 누르면 MATLAB 이 값을 제안함
다만 포화·잡음·모델 오차를 모르므로 제안값에서 시작해 위 순서로 손보는 것이 맞음

1-13. 배의 속도측 — 스프링이 없으면

★ 바뀌는 상자는 플랜트 하나뿐

1-3~1-12 에서 만든 PID 를 그대로 두고 전달함수 자리에 배의 운동방정식을 끼움

- 종방향 운동방정식의 유도과 계수의 출처는 **3주차 1-8 절**임. 본 절은 결과를 받아서 시작함

$$m \dot{u} = X - (X_u + X_{uu}|u|) u \quad m = 211, \quad X_u = 100, \quad X_{uu} = 150, \quad X_{\max} = 500 \text{ N}, \quad T_m = 0.30 \text{ s}$$

⚠ 제어가 낸 값과 실제로 나간 값을 구분함

포화 전 요구값은 X_{raw} , 포화 뒤 실제 값은 X_{sat} 로 적음 (1단계의 $\tau_{\text{raw}} \tau_{\text{sat}}$ 와 같은 자리)

$X_{\max}$ 500 N 은 추진기 250 N $\times$ 2 로 **본 과목이 정한 한계**임. VRX 플러그인 한계는 편당 2353.6 N

- **현상**: 제어가 없이 계단 추력만 주고 Scope 를 세 번 읽음 — 정상상태 속도, 63 % 도달 시간, 속도 천장 (실습 H-3)

① 정상상태 속도 — 유도

1. 속도가 자리를 잡으면 더 변하지 않으므로 $\dot{u} = 0$
2. 남는 것은 $X = (X_u + X_{uu}u) u$ (전진 중이므로 $|u| = u$)
3. 값을 넣어 정리 — $150u^2 + 100u - X = 0$
4. 근의 공식으로 양의 근을 취함

$$u_{ss} = \frac{-X_u + \sqrt{X_u^2 + 4X_{uu}X}}{2X_{uu}}$$

$$5. X = 400 \text{ N} \rightarrow \sqrt{100^2 + 4(150)(400)} = 500 \rightarrow u_{ss} = 400/300 = \mathbf{1.3333 \text{ m/s}}$$

② 속도 천장 — 아무리 밀어도 이보다 빠를 수 없다

6. 같은 식에 $X = X_{\max} = 500 \text{ N}$ 을 넣음 — $\sqrt{310000} = 556.78 \rightarrow u_{ss} = \mathbf{1.5226 \text{ m/s}}$
7. 즉 목표 속도 2.5 m/s 는 이 추력 한계에서는 요구 자체가 불가능함 (1-11 의 와인드업을 만드는 조건)

③ 시상수 — 유도

8. 정상상태 근처의 작은 변화만 봄. $u = u_0 + \delta u$ 로 두고 δu 가 작다고 가정
9. 항력 $X_u u + X_{uu}u^2$ 을 u_0 에서 1차항까지만 전개. 기울기는 $X_u + 2X_{uu}u_0$
10. 정상상태 항이 서로 지워지고 남는 식 — $m \dot{\delta u} = -(X_u + 2X_{uu}u_0) \delta u$
11. 1차계이므로 $T_u = m / (X_u + 2X_{uu}u_0)$
12. $u_0 = 1.3333 \rightarrow T_u = 211/500 = \mathbf{0.422 \text{ s}}$

i 측정한 도달 시간이 손계산 T_u 의 약 2 배인 이유 — 둘 다 맞음

1. 모터 1차 지연 $T_m = 0.30 \text{ s}$ 가 앞에 하나 더 붙어 있음. 손계산은 배만 봄
2. T_u 는 u_{ss} 근처에서 선형화한 값임. $u = 0$ 에서 출발하면 항력이 작아 초반은 빠르지만, 63 % 지점까지의 전체 시간은 선형 1차계보다 길어짐

④ P 만 쓰면 오차가 남고, 출렁이지는 않는다

1. P 제어기는 $X_{\text{raw}} = K_{p,u} (u_d - u)$ 임
2. 정상상태 식에 넣음 — $K_{p,u} (u_d - u_{ss}) = (X_u + X_{uu}u_{ss}) u_{ss}$
3. 정리 — $X_{uu}u_{ss}^2 + (X_u + K_{p,u}) u_{ss} - K_{p,u}u_d = 0$
4. 뜻을 읽으면 — 남는 오차는 그 속도를 유지하는 데 필요한 항력을 $K_{p,u}$ 로 나눈 것

5. $K_{p,u} = 100$, $u_d = 1.0 - 150u^2 + 200u - 100 = 0 \rightarrow u_{ss} = 0.3874$, $e_{ss} = 0.6126$ (실습 I-2 의 측정과 소수 넷째 자리까지 같음)
- $K_{p,u}$ 를 10 배로 해도 오차가 0 이 되지 않음 — **P 만으로는 항력을 이길 수 없음**
- 1단계와 달리 **출렁이지 않음**. 상태가 u 하나뿐이라 2차항이 없기 때문임 (1-3 의 플랜트는 위치와 속도 둘이었음)
- 그래서 속도 루프는 **PI** 를 씀. 남은 항력을 찾아내는 것은 I 의 일임 (1-9)

⑤ D 는 여기서 해롭다

1. 목표 속도가 상수인 동안 $\dot{e} = -\dot{u}$ 이므로 D 항은 $K_{d,u}\dot{e} = -K_{d,u}\dot{u}$
2. 운동방정식에 넣음 — $m\dot{u} = -K_{d,u}\dot{u} + (\text{나머지})$
3. $\dot{u}$ 를 왼쪽으로 모음 — $(m + K_{d,u})\dot{u} = (\text{나머지})$
4. $K_{d,u}$ 가 질량 옆에 붙었음. 배를 무겁게 만드는 것과 같음
- 측정에서도 정착시간이 **4.3 배**로 늘고 추력 변동이 3.3 배가 됨. 좋아지는 항목이 하나도 없어 본 과목은 $K_{d,u} = 0$ 을 씀 (실습 I-3)

⑥ $K_{p,u}$ 의 규모는 단위에서 잡는다

- $K_{p,u}$ 의 단위는 $N \div (m/s)$. 목표 속도를 곱하면 N 이 나옴
- 추력 한계 $\div$ 목표 속도 = $500/1.5 \approx 333$ — 이보다 크게 잡으면 계단이 들어오는 순간 이미 한계에 닿음. 기본 값은 300

1-14. 배의 요측 — 이미 적분기를 갖고 있다

- WAM-V 에는 방향타가 없음. 좌우 추력 차이로 뒹 (탱크·굴착기가 도는 방식)
- 속도 루프에서 헤딩 루프로 넘어갈 때 새로 붙는 블록은 `deg2rad` · `ssa` · `Alloc` 셋뿐임

추력 배분 역변환 — 유도

1. 두 추진기가 만드는 전후력과 요 모멘트 — $X = F_L + F_R$, $N = b(F_L - F_R)$, $b = 1.027135$ m
2. 제어기는 $X \cdot N$ 을 내지만 추진기에 보낼 것은 $F_L \cdot F_R$ 이므로 거꾸로 풀어야 함
3. 두 식을 더함 — $X + N/b = 2F_L$
4. 두 식을 뺀 — $X - N/b = 2F_R$
5. 양변을 2 로 나눔

$$F_L = \frac{X}{2} + \frac{N}{2b}, \quad F_R = \frac{X}{2} - \frac{N}{2b}$$

6. 선회 여유를 계산함. 추진기 한 개의 한계는 250 N 이고 헤딩 실험의 전진 추력은 $X = 300$ N 이므로 각 추진기가 이미 150 N 을 씀
7. 남은 100 N 을 모멘트에 쓰면 $|N| \leq 2b \times 100 = 205$ N·m
- 모멘트 포화를 $N_{\max} = 500$ N·m 로 두었더라도 전진 추력이 크면 실제로 쓸 수 있는 모멘트는 그보다 작음

현상 — 선수각이 멈추지 않는다

- 계단 모멘트를 주면 선회율은 일정해지는데 선수각은 40 초에 $253^\circ \cdot 656^\circ \cdot 986^\circ$ 까지 계속 커짐 (실습 J-3)
- 받아 오는 식은 3주차 1-8 절의 요측 운동방정식임

$$I_z \dot{r} = N - (N_r + N_{rr}|r|) r \quad I_z = 653, \quad N_r = 800, \quad N_{rr} = 800$$

① 정상 선회율 · ② 요 시상수 — 유도

1. 선회율이 자리를 잡으면 $\dot{r} = 0 \rightarrow 800r^2 + 800r - N = 0$
2. 근의 공식 — $r_{ss} = (-N_r + \sqrt{N_r^2 + 4N_{rr}N}) / (2N_{rr})$

3. $N = 300 - \sqrt{1600000} = 1264.9 \rightarrow r_{ss} = 0.2906 \text{ rad/s} = \mathbf{16.65 \text{ deg/s}}$
4. 1-13 ③ 과 같은 방법으로 r_0 근처만 보면 감쇠의 기울기는 $N_r + 2N_{rr}|r_0|$
5. $T_r = I_z / (N_r + 2N_{rr}|r_0|) = 653 / 1264.9 = \mathbf{0.516 \text{ s}}$

③ 왜 선수각은 멈추지 않는가 — 두 줄

6. $\dot{\psi} = r$ 이고 r 이 상수로 자리를 잡음
7. 상수를 시간으로 적분하면 계속 커지므로 ψ 는 멈추지 않음
- 즉 배는 요측에 적분기를 이미 하나 갖고 있음. 속도측과 결정적으로 다른 점임

④ 그래서 헤딩은 P 만으로도 오차가 0 이 된다

1. 목표 선수각에 도달해 멈췄다면 $\dot{\psi} = r = 0$
2. $r = 0$ 이면 요 감쇠 항 $(N_r + N_{rr}|r|)r = 0$
3. 운동방정식에 넣으면 필요한 모멘트도 $N = 0$
4. 제어기가 내는 것은 $N = K_{p,\psi}e$ 이므로 $e = 0$
- 속도측에서는 항력을 이기려고 힘을 계속 내야 했으므로 오차가 남았지만, 요측에서는 멈춘 자리에서 힘이 필요 없음
- 그래서 헤딩 루프의 기본형은 PD 임. I 는 좌우 비대칭 같은 외란이 있을 때만 컴 (실습 K-4)

1-15. 헤딩 계인을 계산으로 정하기 — P-D 극배치

- 1-14 까지는 계인을 키우면 무엇이 달라지는지를 봤음. 본 절은 계인을 **계산으로 정함**
- 필요한 것은 배의 숫자 두 개(I_z, N_r)와 원하는 응답을 나타내는 숫자 두 개(ζ, ω_n)뿐임

$$N_{\text{raw}} = K_{p,\psi} s s a(\psi_{\text{ref}} - \psi) + K_{d,\psi} r, \quad K_{d,\psi} < 0$$

기호	뜻	기본값
ψ_{ref}, ψ	목표·현재 선수각 [rad] (모델 <code>psi_ref</code>)	45° / 측정
r	요각속도 [rad/s], NED (시계 +)	측정
$K_{p,\psi} \cdot K_{d,\psi}$	헤딩 비례·미분 이득 — 미분은 음수가 맞음	800 · -400
N_{max}	요 모멘트 포화 [N·m]	500

D 항을 오차가 아니라 r 에서 얻는 이유

1. 오차는 $e = \psi_{\text{ref}} - \psi$ 이므로 $\dot{e} = \dot{\psi}_{\text{ref}} - r$
2. 목표가 가만히 있는 동안에는 $\dot{\psi}_{\text{ref}} = 0$ 이므로 $\dot{e} = -r$
3. 따라서 $K_d \dot{e}$ 와 $K_{d,\psi} r$ ($K_{d,\psi} = -K_d$) 은 **같은 값**임
4. 다른 것은 목표가 계단으로 바뀌는 순간뿐임 — $\dot{e}$ 는 튀고(1-10 의 미분 킁), r 은 배가 실제로 도는 속도라 튀지 않음
- 실측 (`w04_heading_sign` , 요측만 댄 비선형 모델, $0 \rightarrow 45^\circ$ 계단, 0.05 s 샘플, 포화 $\pm 500 \text{ N·m}$)

D 항	오버슈트 [%]	정착 [s]	포화 전 $ N $ 최대 [N·m]
$K_{d,\psi} r$ ($K_{d,\psi} = -400$, 채택)	0.2	4.7	628
$+K_d \dot{e}$ ($K_d = 400$)	0.2	4.7	6912
부호를 뒤집어 $+400 r$	17.0	10.0	678

- 앞의 둘은 추종 성능이 **완전히 같음** (위 3번 때문). 갈리는 것은 계단 순간의 지령이며 미분 쪽은 포화 한계의 **14 배**까지 치솟음

- 부호를 뒤집으면 회전을 **부추기**는 되먹임이 되어 오버슈트가 17 % 로 커짐

P-D 페루프는 2차계가 된다 — 유도

- 요축 식에서 2차 감쇠 $N_{rr}|r|r$ 를 버림. 계단이 작으면 $|r|$ 이 작아 그 항이 작기 때문임
- 남는 식은 $I_z \dot{r} = N - N_r r$ 이고 $r = \dot{\psi}$ 이므로 $I_z \ddot{\psi} = N - N_r \dot{\psi}$
- 제어기 식을 넣음 (ssa 는 작은 오차에서 그대로 통과) — $I_z \ddot{\psi} = K_{p,\psi}(\psi_{\text{ref}} - \psi) + K_{d,\psi}\dot{\psi} - N_r \dot{\psi}$
- ψ 가 든 항을 왼쪽으로 모음

$$I_z \ddot{\psi} + (N_r - K_{d,\psi}) \dot{\psi} + K_{p,\psi} \psi = K_{p,\psi} \psi_{\text{ref}}$$

- $K_{d,\psi}$ 가 음수이므로 $-K_{d,\psi}$ 는 양수임. D 항이 배가 이미 가진 감쇠 N_r 에 더해짐
- 1-4 의 표준형과 계수를 맞대면

$$\omega_n = \sqrt{\frac{K_{p,\psi}}{I_z}}, \quad \zeta = \frac{N_r - K_{d,\psi}}{2\sqrt{K_{p,\psi} I_z}}$$

- 기본 계인을 넣어 확인 — $\omega_n = \sqrt{800/653} = 1.107 \text{ rad/s}$, $\zeta = 1200/1445.5 = 0.830$

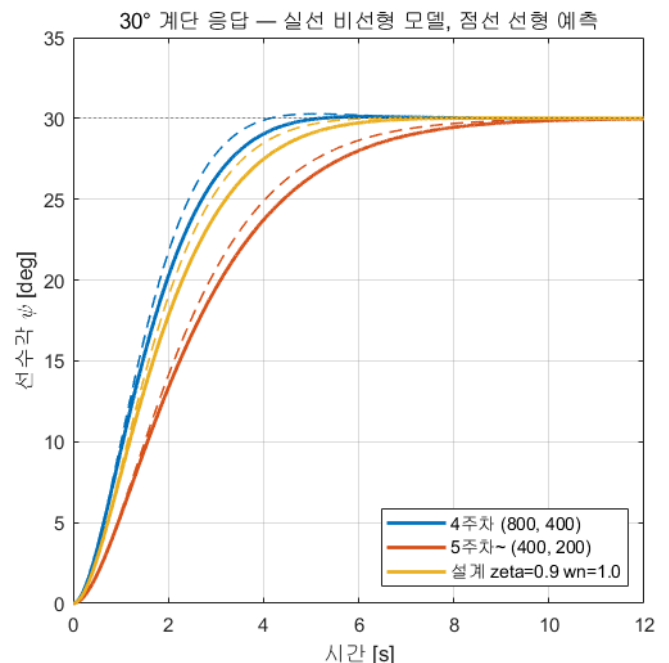
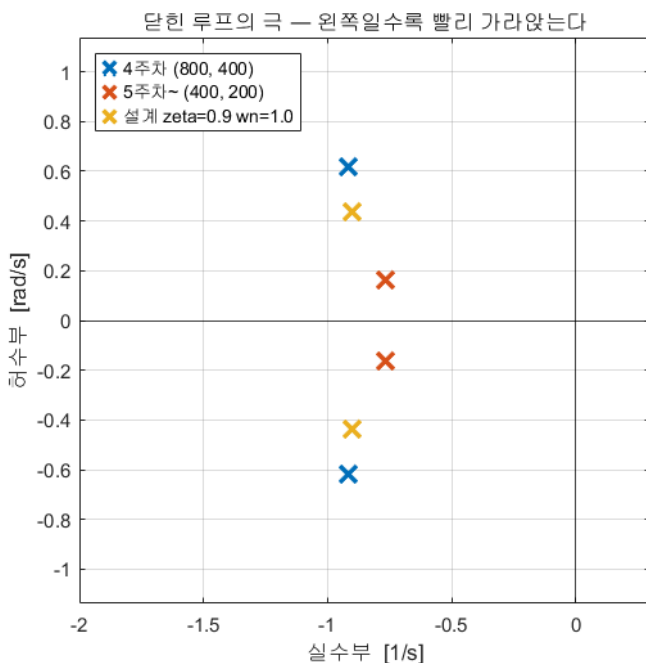
거꾸로 — 원하는 응답에서 계인을 푼다 (극배치)

- 6번의 두 식을 계인에 대해 풀

$$K_{p,\psi} = I_z \omega_n^2, \quad K_{d,\psi} = N_r - 2\zeta \omega_n I_z$$

- $\zeta = 0.9$, $\omega_n = 1.0 \text{ rad/s}$ 를 원하면 — $K_{p,\psi} = 653$, $K_{d,\psi} = 800 - 2(0.9)(653) = -375$
- 분모 $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$ 의 두 근 $s = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$ 이 닫힌 루프의 극임. $\zeta\omega_n$ 을 고르는 것이 곧 극의 자리를 고르는 것이라 **극배치**라 부름
- 실측 (w04_pole_place, 30° 계단, 비선형 요 모델 + 포화 $\pm 500 \text{ N}\cdot\text{m}$, 연속시간)

계인 ($K_{p,\psi}$, $ K_{d,\psi} $)	ω_n [rad/s]	ζ	선형 M_p [%]	비선형 M_p [%]	정착 2 % [s]	$ N $ 최대
(800, 400) — 4주차	1.107	0.83	0.93	0.37	4.27	419
(400, 200) — 5주차 이후	0.783	0.98	0.00	0.00	7.88	209
(653, 375) — 설계 $\zeta = 0.9$, $\omega_n = 1.0$	1.000	0.90	0.15	0.05	5.39	342



- 세 세트 모두 극이 왼쪽 반평면이라 안정하며, 다른 것은 **속도와 넘침의 맞바꿈**임
- 선형 M_p 가 비선형 M_p 보다 큼 — 비선형 모델에는 1번에서 버린 $N_{rr}|r|$ 감쇠가 더 있음. 선형 예측은 보수적

⚠ 계단 크기를 함께 적지 않으면 숫자가 맞지 않음

위 표는 **30° 계단**이고 실습 K 절의 스위치는 VRX 스폰 자세에서 시작하는 **12.3° 계단**임

45° 계단에서는 $K_{p,\psi}e = 800 \times 0.785 = 628 \text{ N}\cdot\text{m}$ 로 포화(± 500)에 걸려 선형 예측이 맞지 않게 됨
극배치는 포화에 닿기 전까지만 약속을 지킴

i $K_{d,\psi}$ 가 양수로 나오면

8번 식에서 $2\zeta\omega_n I_z < N_r$ 이면 $K_{d,\psi}$ 가 양수가 됨 — 원하는 감쇠를 배가 이미 혼자 넘게 주고 있다는 뜻임
예를 들어 $\zeta = 0.9, \omega_n = 0.6$ 이면 $K_{d,\psi} = +95$. 이때는 0 으로 두고 ζ 가 목표보다 커지는 것을 받아들임

1-16. $\pm 180^\circ$ 이음매와 ssa

- **현상:** 목표 -170° , 현재 170° 일 때 두 각을 그대로 빼면 -340° 가 나옴. 실제로 가야 할 거리는 반대 방향으로 20° 임 (실습 L-3 에서 17 배)
- 시계에서 11시 $\rightarrow$ 1시는 2시간이지 10시간이 아님

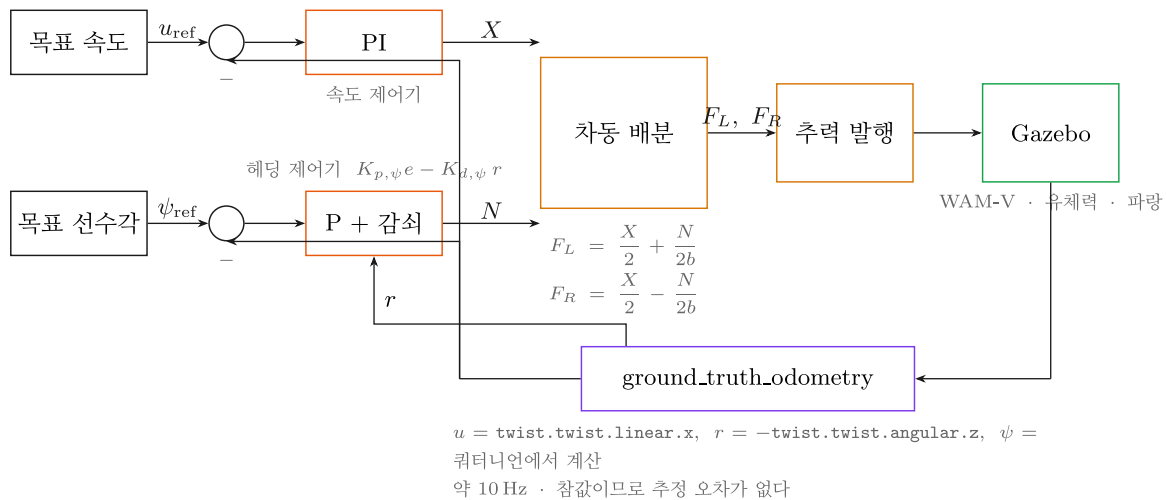
$$\text{ssa}(e) = \text{atan2}(\sin e, \cos e)$$

1. **sin** 과 **cos** 는 360° 마다 같은 값을 돌려줌 — $\sin e = \sin(e \pm 360^\circ)$
2. 따라서 **(sin e, cos e)** 한 쌍은 e 가 몇 바퀴를 돌았는지를 **잊어버림**. 남는 것은 방향뿐임
3. **atan2** 는 그 한 쌍을 다시 각도로 되돌리며 돌려주는 범위가 $(-180^\circ, 180^\circ]$ 임
4. 결과로 **같은 방향을 가리키는 가장 작은 각**이 나옴

```
function e = HeadingErr(psi_ref, psi)
d = psi_ref - psi;
e = atan2(sin(d), cos(d));      % 359도와 1도의 차이는 2도
```

- 헤딩 제어를 쓰는 모든 모델(**w04_3_heading** · **_offline** · **w04_6_wrap**)이 같은 함수를 씀

1-17. 두 루프 겹치기 · 컴퓨터로 옮기기



루프	입력	출력	제어기	근거
속도 루프	$u_{\text{ref}} - u$	전후력 X	PI	1-13 ④ — 항력을 이기려면 I 가 필요
헤딩 루프	$\psi_{\text{ref}} - \psi$	요 모멘트 N	PD	1-14 ④ — 배가 이미 적분기를 가짐

- 두 출력이 1-14 의 배분식을 지나 좌·우 추력이 되고, 각각 `/wamv/thrusters/left/thrust` · `right/thrust` 로 발행됨
- 5주차부터는 이 두 루프 바깥에 **유도(guidance)** 루프가 붙음. 바깥이 낸 명령을 안쪽이 이미 다 따라간 상태여야 하므로 경험칙은 **안쪽이 5~7배 빠르게** — 헤딩 $\omega_n = 1.11$ rad/s 이면 유도는 0.2 rad/s 정도

컴퓨터는 띄엄띄엄 계산한다

- 지금까지의 식은 연속시간이지만 실제 제어기는 T_s 마다 한 번씩만 계산함. 샘플이 너무 느리면 **같은 게인인데도 불안정해짐**
- 1. 샘플 각주파수는 $\omega_s = 2\pi/T_s$
- 2. 실무 기준은 대역폭 ω_B 의 **20배 이상** — $\omega_s \geq 20\omega_B$
- 3. 정리하면 $T_s \leq 2\pi/(20\omega_B) = 0.314/\omega_B$
- ω_B (대역폭)는 **페루프가 따라갈 수 있는 가장 빠른 변화의 정도**이며 2차계에서는 ω_n 과 같은 수준임
- 헤딩 루프 $\omega_B \approx 1.11$ rad/s $\rightarrow T_s \leq 0.28$ s. 본 과목 모델은 0.05 s 이므로 5.6 배 여유가 있음

```
G = tf(1, [1 2 2]);
Gd = c2d(G, 0.05); % 0.5 로 바꾸면 극이 단위원 밖으로 나간다
```

1-18. Simulink 와 ROS 2 는 어떻게 만나는가



- 서로 다른 OS 인데도 통신됨. 이유는 **2주차에 배운 ROS 2 의 DDS 디스커버리**이며, Simulink 쪽에서는 ROS Toolbox 가 ROS 2 노드 역할을 함

블록	역할	라이브러리
Subscribe	토픽 구독 $\rightarrow$ 버스 출력 (포트 1 IsNew , 포트 2 Msg)	<code>ros2lib/Subscribe</code>
Publish	버스 입력 $\rightarrow$ 토픽 발행	<code>ros2lib/Publish</code>
Blank Message	빈 메시지(버스) 생성	<code>ros2lib/Blank Message</code>

```
Blank Message  $\rightarrow$  Bus Assignment  $\rightarrow$  Publish
(std_msgs/Float64 (data 필드에 (토픽으로 내보냄)
빈 껍데기) 숫자를 넣음)
```

- ROS 메시지는 **구조체(버스)** 라서 숫자를 바로 못 넣음. 빈 메시지를 만들고 $\rightarrow$ 필드를 채우고 $\rightarrow$ 발행하는 3단계 임
- 필드가 많은 메시지에서는 이 방식이 유일하게 통함. 블록별 사용법은 [A1 Simulink 기초](#)

IsNew 를 Display 에 연결해 두면 진단이 쉬움

값이 계속 0 이면 토픽이 안 들어오는 것임. 연결 문제를 즉시 알 수 있음

1-19. 속도 u 얻기 · 페이싱 · VRX 경량 기동

★ 제어를 만들려면 지금 몇 m/s 로 가는지를 알아야 함

그런데 VRX 의 기본 센서(GPS · IMU)는 속도를 직접 주지 않음

방법	원리	문제
GPS 위치 미분	위치 차분 $\div$ 시간	GPS 잡음이 증폭돼 속도가 요동침 (1-10 과 같은 이유)
IMU 가속도 적분	가속도를 시간 적분	드리프트 — 시간이 갈수록 값이 흘러감
EKF 융합	GPS + IMU 를 칼만필터로	만드는 데 시간이 오래 걸림. 튜닝도 필요
ground truth odometry	Gazebo 가 참값을 직접 알려줌	실제 배에는 없음 (시뮬레이션 전용)

- 본 과목은 참값을 씀. 제어를 처음 만들 때 추정 오차와 제어 오차가 섞이면 무엇이 문제인지 알 수 없기 때문임
- 먼저 참값으로 제어 성능의 기준선을 만들고, 추정기로 바꾸는 것은 그다음 문제임. 이것이 제어 설계의 일반적인 순서임
- 토픽은 `/wamv/sensors/position/ground_truth_odometry` (`nav_msgs/Odometry` , 10 Hz, `map` → `wamv/base_link`)

ROS 필드	ROS 에서의 뜻	제어식의 기호	바꾸는 법
<code>twist.twist.linear.x</code>	선체 앞쪽 속도	u 전후 속도	$u = \text{linear.x}$ (그대로)
<code>twist.twist.linear.y</code>	선체 좌현 쪽 속도	v 좌우 속도 (우현 +)	$v = -\text{linear.y}$
<code>twist.twist.angular.z</code>	ω_z — 반시계 +	r 요각속도 (시계 +)	$r = -\omega_z$
<code>pose.pose.orientation</code>	쿼터니언 → ψ_{ENU}	ψ 선수각 (북에서 시계)	$\psi = 90^\circ - \psi_{\text{ENU}}$

⚠ ROS 가 주는 값에 제어식의 글자를 바로 붙이지 않음

ROS 의 선체축은 x_b 선수 y_b 좌현 z_b 위(FLU), 제어식은 x_b 선수 y_b 우현 z_b 아래(FRD)
 y_b 와 z_b 가 뒤집혀 있으므로 옆 속도와 요각속도는 부호가 반대 (3주차 ENU→NED)
선수각 변환식 $\psi_{\text{NED}} = \pi/2 - \psi_{\text{ENU}}$ 의 양변을 미분하면 $r = -\omega_z$ 가 그대로 나옴
1-15 의 $K_{d,\psi}r$ 이 감쇠가 되려면 r 이 선수각과 같은 방향(시계 +)이어야 함

! twist 는 선체 기준

ROS 규약상 twist 는 `child_frame_id` (= `wamv/base_link`) 기준이므로 `linear.x` 가 곧 전후 속도 u 임
2026-09-03 실측: 좌우 추력 200 N 을 8초 주었더니 `linear.x = 1.33` , `linear.y ≈ 0.002` → 앞으로만 가고 옆으로는 안 감

실시간 페이싱 — 안 켜면 배가 안 움직인다

- Simulink 는 기본적으로 최대한 빨리 계산함. 20초짜리 시뮬레이션이 0.3초 만에 끝나기도 함
- 그동안 Gazebo 는 실제 시간으로 흐르므로 추력이 순식간에 다 나가고 모델이 멈춤. 배는 거의 안 움직임
- 고치는 법 — 톨스트립 Run 옆 화살표 → **Simulation Pacing** → *Enable pacing to slow down simulation* 체크, 비율은 실측 RTF

```
rtf = 0.85; % gz topic -e -t /stats 로 잔 값
set_param('W04_1_straight','EnablePacing','on','PacingRate',num2str(rtf));
```

★ 비율은 1 이 아니라 실측 RTF

Gazebo 가 RTF 0.85 로 도는데 비율 1 로 돌리면 모델의 1초가 시뮬레이터의 0.85초에 대응 → **모델이 앞서 나감**

꺾이는 꺾치는데 시간이 어긋나 이격이 시간에 비례해 커짐. `W04_step_compare` 는 이 측정을 자동으로 함

```
gz topic -e -t /stats -n 1 | grep real_time_factor
```

⚠ RTF 는 고정값이 아님

기준 환경에서 같은 월드를 5초 간격으로 재면 **0.46 ~ 0.91** 까지 흔들렸음 (2026-09-15 실측)

다른 프로그램·화면 녹화·브라우저가 돌고 있으면 더 심해짐. **비교 실험은 같은 조건에서 연속으로 돌릴 것**

VRX 는 경량 모드로 띄운다

- 3주차 2-3 에서 만든 `W03_vrx_lite/run_vrx.sh` 를 그대로 씀

항목	값
선체	<code>W03_vrx_lite/wamv_lite.urdf</code> — GPS · IMU · 참값 오도메트리 만. 카메라·LiDAR 없음
렌더러	<code>--render-engine-gui ogre</code> (그림을 그리는 센서가 없으므로 서버 엔진은 지정하지 않음)
월드	<code>sydney_regatta</code> (기본). 바꿀 때 <code>WORLD=...</code>
명령만 보기	<code>DRY=1 bash run_vrx.sh</code>

- 카메라 3대 + LiDAR 를 켜는 `full` 모드는 **9주차(VRX 심화)** 몫이며 이번 주차에서는 쓰지 않음
- 참값 오도메트리가 lite URDF 에 **이미 켜져 있음**. urdf 를 손으로 고칠 필요가 없음 (실습 N 절)